

基于规则增强强化学习在大型FPS游戏地图中训练交互智能体

张晨^{1,2}, 胡欢², 周远², 曹启阳², 刘若尘², 魏文雅², Elvis S. Liu², *¹ 中国, 合肥, 中国科学技术大学软件工程学院² 腾讯游戏 zhangchenzc@mail.ustc.edu.cn, {luckyhu, ariellezhou, hughyycao, ruochenliu, wenyawei, elvissyliu}@tencent.com

摘要——在竞技游戏领域, 三维第一人称射击游戏 (3D FPS游戏) 广受欢迎, 推动了游戏人工智能系统的发展以提升游戏体验。然而, 在实际场景中部署游戏AI仍面临挑战, 尤其是在大规模且复杂的FPS游戏中。本文聚焦于腾讯游戏开发的在线多人竞技3D FPS游戏《Arena Breakout》中游戏AI的实际部署。我们提出了一种新颖的游戏AI系统, 名为私人军事公司智能体 (PMCA), 它能够在大型游戏地图中与玩家互动并交战, 同时利用周围地形提供的战术优势。

为了解决现代3D FPS游戏中的导航与作战挑战, 我们提出了一种结合导航网格与射击规则的深度强化学习方法 (NSRL)。导航网格的整合增强了智能体的全局导航能力, 而射击行为则采用基于规则的方法进行控制以确保可控性。NSRL利用深度强化学习模型预测何时启用导航网格, 从而为游戏AI带来多样化的行为模式。此外, 还采用了针对类人行为的定制化奖励机制, 使PMCA的行为与人类玩家保持一致。

索引术语——游戏AI、深度强化学习、导航网格、射击规则、自我博弈、规则增强。

I. 引言

三维第一人称射击游戏在竞技游戏领域获得了极高的人气。随着这类游戏从早期的《Maze War》和《半条命》发展到近期的《Apex Legends》《CS: GO》和《Valorant》, 为FPS游戏开发智能人工智能系统的兴趣日益浓厚。基于行为树的传统决策方法在复杂3D环境中探索所有可能决策方面已显不足。为克服这一局限, 深度强化学习作为设计3D FPS游戏AI的灵活替代方案被引入。

然而, 尽管深度强化学习智能体在技术上取得了显著进展, 展现了强大能力, 但在FPS游戏中实际部署游戏AI仍面临挑战。现有的用于训练和测试AI智能体的环境通常游戏地图较小、持续时间有限, 这与现代FPS游戏的规模和复杂性不相符。

*通讯作者

979-8-3503-5067-8/24/\$31.00 ©2024 IEEE

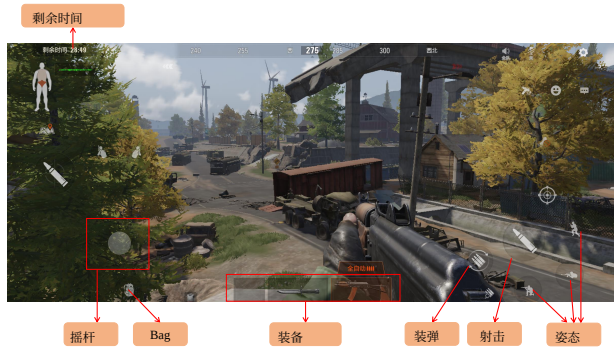


图1. Arena Breakout的界面

例如, 在VizDoom这样的环境中, 每场比赛的持续时间通常约为100帧。此外, 3D FPS游戏环境的复杂性, 包括物理建模的地形和物体, 给AI智能体的环境感知和全局导航带来了困难。更关键的是, 游戏AI需要在全局导航与战斗交战之间取得平衡, 这提出了很高的要求, 要求同时进行宏观层面的导航决策和微观层面的战斗决策。同时, 还需要确保游戏AI既具备竞技实力, 又表现出类人行为。

在本文中, 我们试图解决这些问题, 并专注于游戏AI在由腾讯游戏开发的一款名为Arena Breakout的在线多人3D FPS游戏中的实际部署。在Arena Breakout中, 玩家控制一个第一人称视角的游戏角色, 目标是限时内抵达指定地点以撤离。在整个游戏过程中, 玩家通过与躲避或消灭其他玩家或游戏AI, 实现撤离。

为了提升玩家的游戏体验, 我们提出了一种名为私人军事公司智能体 (PMCA) 的新型游戏AI系统, 该系统可在大型游戏地图中进行交互。PMCA主要部署在涉及高水平玩家的对局中。当PMCA在地图任意位置遭遇玩家时, 它会利用周围地形获取战术优势, 发起攻击并与玩家交战。为了实现这些目标, PMCA不仅要解决现代3D FPS

Training Interactive Agent in Large FPS Game Map with Rule-enhanced Reinforcement Learning

Chen Zhang^{1,2}, Huan Hu², Yuan Zhou², Qiyang Cao², Ruochen Liu², Wenyu Wei², Elvis S. Liu^{2,*}
¹ School of Software Engineering, University of Science and Technology of China, Hefei, China
² Tencent Games
zhangchenzc@mail.ustc.edu.cn,
{luckyhu, ariellezhou, hughyycao, ruochenliu, wenyawei, elvissyliu}@tencent.com

Abstract——In the realm of competitive gaming, 3D first-person shooter (FPS) games have gained immense popularity, prompting the development of game AI systems to enhance gameplay. However, deploying game AI in practical scenarios still poses challenges, particularly in large-scale and complex FPS games. In this paper, we focus on the practical deployment of game AI in the online multiplayer competitive 3D FPS game called Arena Breakout, developed by Tencent Games. We propose a novel gaming AI system named Private Military Company Agent (PMCA), which is interactable within a large game map and engages in combat with players while utilizing tactical advantages provided by the surrounding terrain.

To address the challenges of navigation and combat in modern 3D FPS games, we introduce a method that combines navigation mesh (Navmesh) and shooting-rule with deep reinforcement learning (NSRL). The integration of Navmesh enhances the agent's global navigation capabilities while shooting behavior is controlled using rule-based methods to ensure controllability. NSRL employs a DRL model to predict when to enable the navigation mesh, resulting in a diverse range of behaviors for the game AI. Customized rewards for human-like behaviors are also employed to align PMCA's behavior with that of human players.

Index Terms——game AI, deep reinforcement learning, navigation mesh, shooting rules, self-play, rule-enhanced.

I. INTRODUCTION

First-person shooter (FPS) games in 3D have gained immense popularity in the competitive gaming realm. As these games have evolved from early titles like Maze War and Half-Life to more recent ones such as Apex Legends, CS: GO, and Valorant, there has been a growing interest in developing intelligent AI systems for FPS games. Traditional decision-making approaches based on behavior trees (BT) have proven inadequate in exploring all possible decisions within complex 3D environments. To address this limitation, deep reinforcement learning (DRL) has been introduced as a flexible alternative for designing game AI in 3D FPS games.

However, despite significant advancements that have shown the power of DRL agents [1, 2, 3, 4, 5], the practical deployment of game AI in FPS games still faces challenges. Existing environments used for training and testing AI agents often have small game maps and limited game duration, which do not align with the scale and complexity of modern FPS games.

*Corresponding author

979-8-3503-5067-8/24/\$31.00 ©2024 IEEE

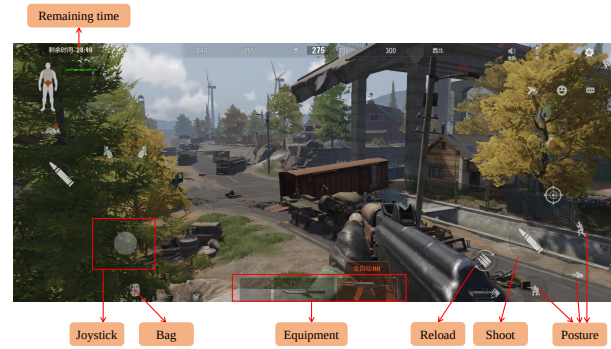


Fig. 1. The interface of Arena Breakout

For example, in environments like VizDoom [1], the duration of each match is typically around 100 frames. Additionally, the complexity of 3D FPS game environments, with physically modeled terrain and objects, poses difficulties for environment perception and global navigation for AI agents. Furthermore, balancing global navigation and combat engagement within the game AI presents significant demands, requiring simultaneous macro-level navigation and micro-level combat decision-making. There is also a need to ensure that the game AI exhibits both competitive strength and human-like behavior.

In this paper, we try to solve these issues and focus on the practical deployment of game AI in an online multiplayer 3D FPS game developed by Tencent Games called Arena Breakout. Players in Arena Breakout control a first-person perspective game character and aim to reach a designated location for evacuation within a limited timeframe. Throughout the game, players interact with other players or game AI, either by evading or eliminating them, to achieve evacuation.

To enhance players' gaming experience, we propose a novel game AI system named Private Military Company Agent (PMCA) which is interactable in a large game map. PMCA is primarily deployed in matches involving high-level players. When PMCA encounters players at any location on the map, it will initiate attacks and engage in combat with the player, utilizing the surrounding terrain to gain tactical advantages. In order to achieve the objectives, PMCA must not only address the challenges of navigation and combat in modern 3D FPS

游戏中导航与战斗的挑战，还需在平均时长超过四十分钟的大型对局中有效运作。此外，PMCA需要约束自身行为，以避免让玩家感到困惑。

为了应对这些挑战，我们提出了一种新颖的方法，将导航网格（Navmesh） [6] 和射击规则与强化学习（NSRL）相结合。集成导航网格增强了智能体的全局导航能力，同时通过基于规则的方法执行开火和射击行为以提升可控性。与以往直接将规则融入程序的方法不同，NSRL采用更巧妙的方式，利用深度强化学习模型来预测是否启用导航网格。仅当深度强化学习模型判断合适时，才会做出切换到导航网格的决策，从而为游戏AI带来更多样化的行为。此外，针对类人行为定制的奖励机制被用来确保PMCA的行为与人类玩家保持一致，进一步强化其类人行为。

总体而言，我们构建了Arena Breakout的马尔可夫决策过程（MDP），并采用PPO算法 [7] 来更新策略。实验结果表明PMCA在全局导航能力和行为多样性方面表现出色。

本文的贡献如下：

- 我们提出了一种基于规则增强强化学习的新型游戏AI系统，将导航网格与射击规则集成到深度强化学习（DRL）中，以提升深度强化学习智能体的性能。该设计允许智能体在实现全地图全局导航的同时，还能精确瞄准并射击目标。

- 我们将该系统部署于Arena Breakout中，并与玩家进行长期交互，这标志着深度强化学习在实际应用中的一个重要里程碑。

II. 符号与背景介绍

A. Arena Breakout

《Arena Breakout》是一款由腾讯游戏基于虚幻引擎4开发的3D第一人称射击游戏，其界面如图1所示。截至撰写本文时，该游戏在全球拥有超过8000万注册玩家。与上述游戏不同，《Arena Breakout》不仅保留了原有的射击游戏逻辑，还增加了更多战术内容。玩家需要在有限时间内争夺资源，并抵达指定位置完成撤离。游戏目标是生存下来，同时尽可能获取更多资源（武器、装备等），将其存入保险箱，并到达撤离点完成撤离。每位玩家都有一个保险箱，箱内资源在死亡时不会丢失，而箱外物品则会丢失。因此，游戏玩法的核心在于玩家如何在保持存活的情况下获取最大资源量。由于基于虚幻引擎4开发，《Arena Breakout》拥有

复杂的3D建模和精细的地图，其中大部分物体都进行了物理建模。玩家在《Arena Breakout》中需要复杂的操作，因为玩家需要在瞄准和射击目标的同时控制方向和移动。游戏还包括蹲伏、匍匐和跳跃等改变玩家姿态的动作。复杂的环境和多样化的操作增加了游戏的可重玩性，同时也提高了训练游戏AI的难度。

B. 状态空间与动作空间

为了将强化学习算法应用于Arena Breakout，我们需要构建游戏的马尔可夫决策过程（MDP）。我们将Arena Breakout分解为状态空间和动作空间。

在Arena Breakout的状态空间中，我们设计了多种与智能体相关的信息源，包括标量游戏变量和通过射线检测获取的感知信息。标量游戏变量，如智能体的位置、旋转角度、朝向以及战斗状态（如生命值和交火状态），通过虚幻引擎4提供的API直接获取。这些变量经过数学处理并综合成五类基础信息，用于输入神经网络（表I）。每类信息对智能体的决策和行为产生不同影响。基于射线检测的感知信息也通过虚幻引擎4实现。智能体从其位置向游戏内实体发射射线，并捕获实体的相关细节，例如实体表面法向量。因此，智能体可以利用这些射线确定实体相对于自身的距离和位置。我们进一步处理这些信息以生成深度图并封装圆形射线，随后将其作为神经网络的输入。

《Arena Breakout》的操作通过一个决定方向的摇杆结合不同的动作按钮来实现。因此，在动作空间中，我们模仿了人类玩家的左右手操作。基于欧拉旋转角，我们将可执行动作分为九个动作头（见表I）。每个动作头由一个独热向量表示，指示该动作的执行状态。开火决定是否发起开火动作。枪偏航和枪俯仰表示如何调整枪管位置。移动类型决定移动模式，例如奔跑或行走。路径类型指示寻路方法，模型将决定是使用原子运动还是导航网格移动。运动方向表示移动方向，即一个固定角度的源运动方向。姿势类型表达当前角色的姿势，如蹲伏、匍匐、跳跃等。瘦型指示是否执行侧身动作。特殊类型代表游戏中的特殊操作，例如开镜瞄准（ADS）或其他特定动作。

games but also perform effectively in large-scale matches with an average duration exceeding forty minutes. Furthermore, PMCA needs to constrain its behavior so that does not confuse the players.

To tackle these challenges, we propose a novel method that combines Navigation Mesh (Navmesh) [6] and shooting rules with reinforcement learning (NSRL). The integration of Navmesh enhances the agent’s global navigation capabilities while considering firefights and shooting behavior is executed using rule-based methods to enhance controllability. Unlike previous approaches that directly incorporate rules into the program, NSRL takes a more subtle approach by using a DRL model to predict whether to enable the navigation mesh. The decision to switch to the navigation mesh is made only when the DRL model determines it is appropriate, resulting in a more diverse range of behaviors for the game AI. Additionally, customized rewards for human-like behaviors are employed to ensure that PMCA’s behavior aligns with human players, further enhancing its human-like behavior.

Overall, we construct the Markov decision process (MDP) of Arena Breakout and employ the Proximal Policy Optimization (PPO) [7] algorithm to update the policy. Experimental results show the ability of PMCA in global navigation and the diversity of behavior.

The contributions of this paper are as follows:

- We propose a novel game AI system based on rule-enhanced reinforcement learning, integrating Navmesh and shooting rules into deep reinforcement learning (DRL) to enhance the performance of the DRL agent. This design allows the agent to balance global navigation across the entire map while accurately aiming and firing at targets.
- We deploy the system in Arena Breakout and engage in long-term interactions with players, representing a significant milestone for the practical application of DRL.

II. NOTATION AND BACKGROUND

A. Arena Breakout

Arena Breakout is a 3D first-person shoot game developed by Tencent Games based on Unreal Engine 4. Its interface is shown in fig. 1. At the time of writing this paper, it has more than 80 million registered players worldwide. Different from the games mentioned above, Arena Breakout not only retains the original shooting game logic but also adds more tactical content. Players need to compete for resources and reach a designated location for evacuation within a limited timeframe. The objective of the game is to survive while obtaining as many resources (weapons, equipment, etc.) as possible, storing them in a secure box, and reaching the evacuation point to complete the evacuation. Each player has a secure box, and resources stored in the box will not be lost upon death, while items outside the box will be lost. Therefore, the core of the gameplay is how players can obtain the maximum amount of resources while staying alive. Owing to developed by Unreal Engine 4, Arena Breakout features

complex 3D modeling and detailed maps where most objects are physically modeled. The player in Arena Breakout requires intricate operation, as players need to control their direction and movement while aiming and firing at targets. The game also includes actions such as crouching, crawling, and jumping to change the player’s posture. The complex environment and diverse operations add more replayability to the game while increasing the difficulty of training the game AI.

B. State Space and Action Space

In order to apply reinforcement learning algorithms to Arena Breakout, we need to construct the Markov Decision Process (MDP) for the game. We decompose Arena Breakout into the state space and the action space.

In the state space of the Arena Breakout, we have devised multiple information sources concerning the agent, encompassing scalar game variables and perception information derived from raycast detection. The scalar game variables, including the agent’s position, rotation angle, orientation, and combat status (such as health and engagement in combat), are directly acquired through the APIs provided by Unreal Engine 4. These variables are subjected to mathematical processing and synthesizing them into five distinct categories of fundamental information for input into the neural network (Table I). Each category of information influences the agent’s decision-making and behavior differently. The perception information based on raycast detection is also implemented through the Unreal Engine 4. The agent emits rays from its position towards entities within the game and captures pertinent details, such as the entity’s surface normal vector. Consequently, the agent can ascertain the distance and position of the entity relative to itself utilizing these rays. We further process this information to generate depth maps and encapsulate the circular ray, which are subsequently employed as inputs for the neural network.

The operation of Arena Breakout is achieved through a joystick that determines the direction combined with different action buttons. Therefore, in the action space, we imitate the left and right-hand operations of human players. Based on Euler rotation angles, we divide the executable actions into nine action heads (table I). Each action head is represented by a one-hot vector that indicates the execution state of that action. Fire determines whether to initiate the firing action. Gun_yaw and Gun_pitch represent how to adjust the position of the gun barrel. Move_type determines the movement mode, such as running or walking. Path_type indicates the pathfinding method, where the model determines whether to use atomic movement or Navmesh movement. Motion represents the movement direction, indicating a source motion at a fixed angle. Posture_type expresses the current character’s posture, such as crouching, prone, jumping, etc. Lean_type indicates whether to perform a leaning action. Special_type represents special operations in the game, such as aiming down sights (ADS) or other specific actions.

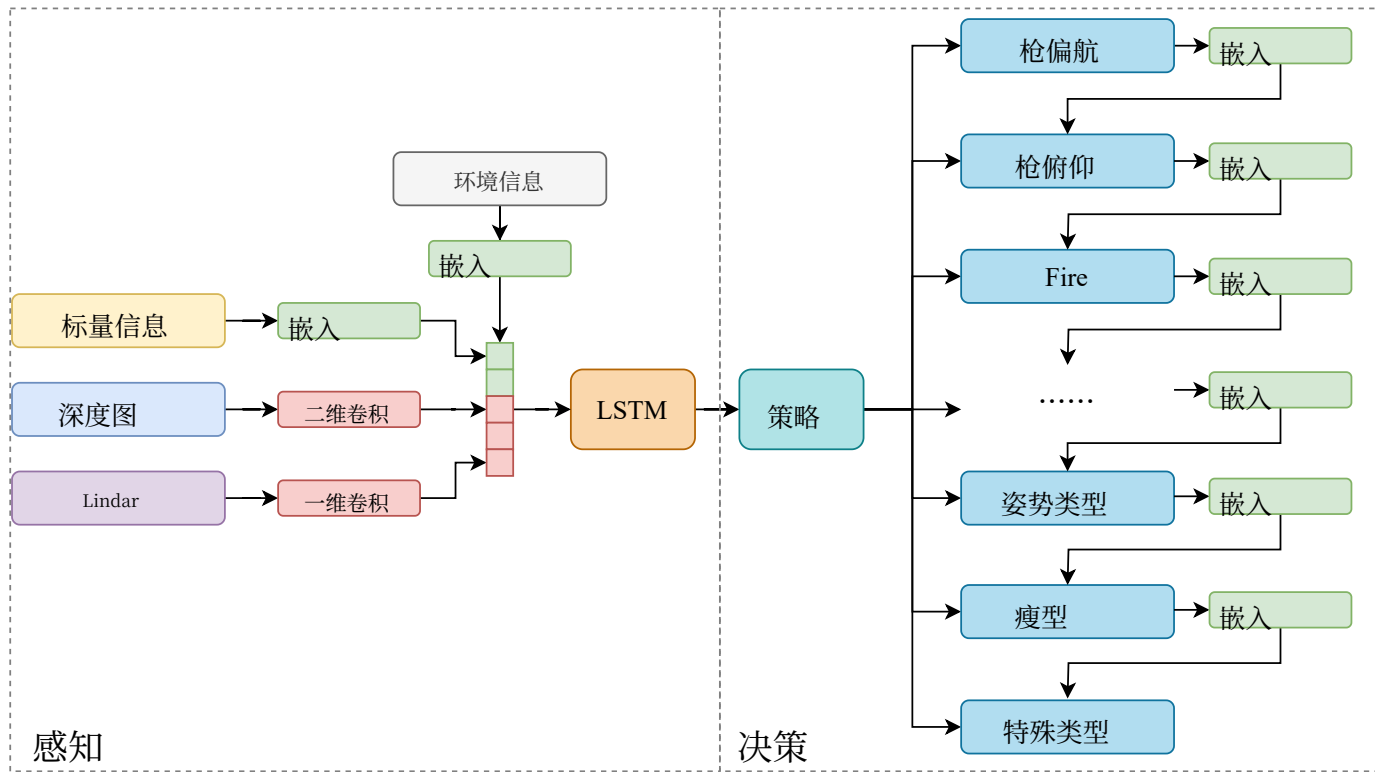


图2. PMCA框架。

状态	Type	Dim
基本信息	浮点数	[1,124]
对手信息	浮点数	[1,99]
环境信息	浮点数	[1,301]
深度图	浮点数	[40,80]
Lindar[8]	浮点数	[144,3]
动作	Type	Dim
Fire	int	[2]
枪偏航	int	[13]
枪俯仰	int	[8]
移动类型	int	[4]
路径类型	int	[4]
运动方向	int	[17]
姿势类型	int	[4]
瘦型	int	[3]
特殊类型	int	[3]

表1 ARENA BREAKOUT的状态空间和动作空间

C. 强化学习

在标准强化学习设置中，智能体与环境交互并接收关于状态的完整感知信息。《Arena Breakout》的马尔可夫决策过程（MDP）被表述为一个五元组 $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$ ，其中 $\mathcal{S}$ 和 $\mathcal{A}$ 是状态空间和动作空间； $r: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 和 $\mathcal{P}: \mathcal{S} \times \mathcal{A} \rightarrow \Delta_{\mathcal{A}}$ 是奖励函数和转移概率分布， $r(s, a) \in [R_{min}, R_{max}]$ 和 $P(\cdot|s, a)$ 是在状态s下执行动作a获得的奖励和下一状态概率； γ 是折扣因子。在每个时间步t，智能体获得一个状态 $s_t \in \mathcal{S}$ 作为环境的当前状态。智能体根据策略 $\pi(a_t|s_t)$ ，在给定状态 s 的情况下预测一个动作 $a_t \in \mathcal{A}$ 。动作执行后，智能体获得奖励 $r(s_t, a_t)$ 。状态值函数

$V_{\pi}(s) = \mathbb{E}_{\pi}[\sum \gamma^k R_{t+k+1} | S_t = s]$ 和动作价值 $Q_{\pi}(s, a) = \mathbb{E}_{\pi}[\sum \gamma^k R_{t+k+1} | S_t = s, A_t = a]$ 始终被用来优化策略。优势函数 $A(s, a) = Q(s, a) - V(s)$ 用于衡量动作 a 相对于平均质量的优劣。

强化学习算法的目标是找到一个最优策略 π^* ，以最大化折扣累积奖励的期望：

$$\pi^* = \arg \max_{\pi} \mathbb{E}[\sum_{t=0}^n \gamma^t r(s_t, a_t)] \quad (1)$$

目前已提出多种深度强化学习算法，其中一些采用基于值的方法来寻找最优值函数，例如深度Q网络（DQN）[9]，而另一些则利用策略梯度通过奖励[10]的梯度来优化策略。Schulman等人 [7] 借助重要性采样提升了收敛效率，数据复用的特点也使得分布式算法的应用成为可能。我们的工作基于PPO。

III. 方法

现代三维第一人称射击游戏通常具有复杂环境和大规模地图的特点。在特定位置进行精确定位的需求给全局导航带来了挑战。在《Arena Breakout》中，PMCA需要追捕地图上遇到的任何玩家，这进一步凸显了精确导航的必要性。另一方面，与玩家进行交火也带来了

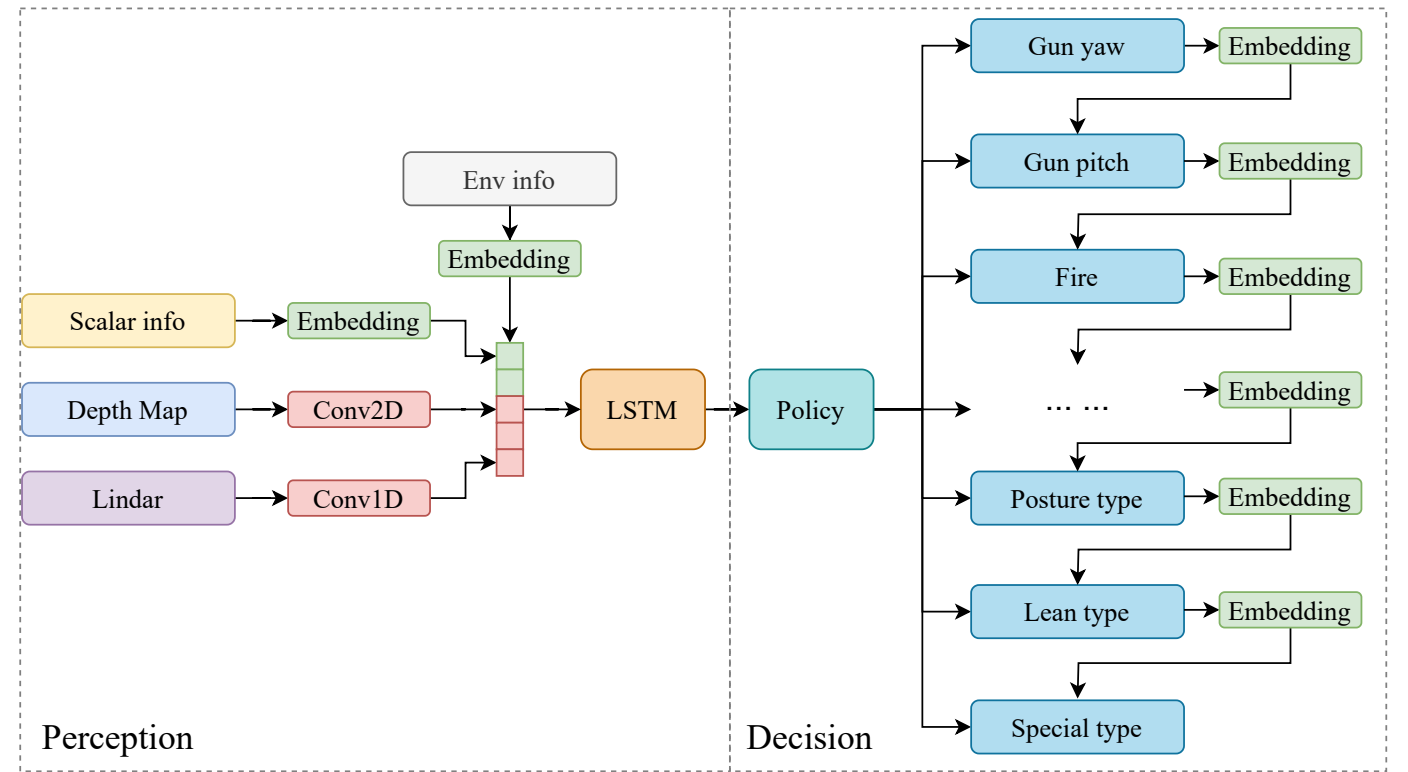


Fig. 2. Framework of PMCA.

State	Type	Dim
Basic_info	float	[1,124]
Opponent_info	float	[1,99]
Env_info	float	[1,301]
Depth Map	float	[40,80]
Lindar[8]	float	[144,3]
Action	Type	Dim
Fire	int	[2]
Gun_yaw	int	[13]
Gun_pitch	int	[8]
Move_type	int	[4]
Path_type	int	[4]
Motion	int	[17]
Posture_type	int	[4]
Lean_type	int	[3]
Special_type	int	[3]

TABLE I
STATE SPACE AND ACTION SPACE OF ARENA BREAKOUT

C. Reinforcement Learning

In a standard reinforcement learning setting, the agent interacts with the environment and receives full perceptual information about the state. The Markov decision process (MDP) of Arena Breakout is formulated as a 5-tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$, where $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces; $r: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ and $\mathcal{P}: \mathcal{S} \times \mathcal{A} \rightarrow \Delta_{\mathcal{A}}$ are the reward function and transition probability distribution, with $r(s, a) \in [R_{min}, R_{max}]$ and $P(\cdot|s, a)$ being the reward and the next state probability of taking action a in state s ; γ is discount factor. In each step t , the agent gets a state $s_t \in \mathcal{S}$ as the current state of the environment. The agent predicting an action $a_t \in \mathcal{A}$ given the state s with the policy $\pi(a_t|s_t)$. Agent gets a reward $r(s_t, a_t)$ after the action is executed. The state value function

$V_{\pi}(s) = \mathbb{E}_{\pi}[\sum_{k=0}^T \gamma^k R_{t+k+1} | S_t = s]$ and the action value function $Q_{\pi}(s, a) = \mathbb{E}_{\pi}[\sum_{k=0}^T \gamma^k R_{t+k+1} | S_t = s, A_t = a]$ are always leveraged to optimize policy. The advantage function $A(s, a) = Q(s, a) - V(s)$ is used to measure the quality of action a compared to the average quality.

The goal of the RL algorithm is to find an optimal policy π^* to maximize the expectation of the discounted cumulative reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}[\sum_{t=0}^n \gamma^t r(s_t, a_t)] \quad (1)$$

Several DRL algorithms have been proposed, some use value-based methods to find an optimal value function such as Deep Q-net work (DQN)[9], and others use policy gradient to optimize policy with the gradient of reward[10]. Schulman et al. [7] utilizing importance sampling to improve convergence efficiency, the characteristic of data reuse also enables the application of distributed algorithms. Our work is based on PPO.

III. METHODS

Modern 3D FPS shooter games typically exhibit characteristics such as complex environments and large-scale maps. The need for precise positioning in specific locations poses challenges for global navigation. In Arena Breakout, PMCA is required to pursue any encountered player on the map, further emphasizing the need for accurate navigation. On the other hand, engaging in combat with players presents

挑战，因为现代三维第一人称射击游戏通常模拟枪械后坐力，从而产生逼真的子弹弹道。这对智能体来说是一项训练挑战，因为它必须学习如何控制子弹弹道以准确命中对手。本节将讨论如何在 1000×400 （单位：UE4中的米）地图上训练一个全局交互游戏，以及为应对这两个挑战所采用的方法。这些方法的结果将在第四节中展示。

A. 框架

我们的框架如图2所示。在每个步骤中，游戏的基本信息会分别输入到特征提取模块中。所有标量信息通过前馈网络进行特征提取。深度图和Lindar [8] 输入到特征提取模块。通过利用二维卷积感知深度图中物体的轮廓特征，并采用一维循环卷积提取周围地形特征，我们实现了基于多模态融合的环境感知方法，该方法将每个特征提取模块的特征与连接后的游戏环境信息相融合。

为了赋予模型记忆能力，我们采用了长短期记忆网络（LSTM）。经LSTM过滤后的特征随后输入到策略网络和价值网络，分别用于动作预测和价值计算。如第二节B部分所述，我们基于人类操作将动作空间划分为九个动作头。每个动作头由一个独热向量表示，用以指示相应动作的执行状态。各动作头之间存在一定的依赖关系。例如，根据游戏本身的限制，模型不应同时预测射击和移动。为解耦动作间的依赖关系，我们未采用动作头并行预测的方式，而是利用带有自回归嵌入的分层动作掩码 [11] 按顺序输出动作头。

在典型的强化学习任务中，动作掩码用于在特定状态下屏蔽相应动作，以提升探索效率。在我们的策略网络中，所有动作头按顺序生成。除第一个动作头外，每个后续动作头的输出由前一个动作的嵌入和策略网络的输出共同决定。通过自回归嵌入，策略隐式地学习了前后动作之间的依赖关系，并将这种依赖逐层传播。在动作预测过程中，策略可以暂时屏蔽后续层中与当前动作冲突的动作。同时，策略还会根据前序动作决定是否屏蔽当前层中的动作。通过这种方式，我们利用自回归预测实现了分层动作掩码，在确保动作合规性的同时提升了探索效率。当所有动作头完成输出后，这些动作将被发送至游戏客户端执行。

B. 导航网格与射击规则增强的强化学习

为了解决在 1000×400 （单位：UE4中的米）地图上的全局导航以及游戏中遭遇敌人时的开火问题，我们采用了导航网格与射击规则增强强化学习（NSRL）方法。该方法结合了导航网格（Navmesh）与原子射击规则的集成，以提升游戏AI的性能。

全局导航。在全局导航中，直接将导航网格纳入程序会导致游戏AI产生基于规则的行为，从而牺牲深度强化学习模型所提供的多样性。因此，在NSRL中采用了一种更柔和的方式。使用导航网格的决策权被委托给深度强化学习模型，使模型能够预测是否启用导航网格。通过将导航网格设置为可预测动作，NSRL既保持了全局导航能力，又保留了深度强化学习模型为游戏AI提供的多样性。

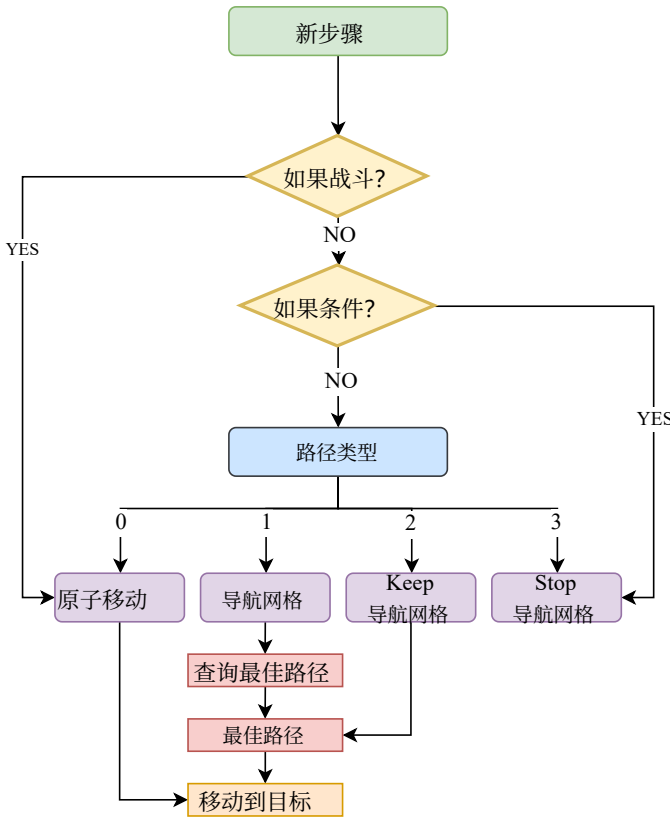


图3. 导航网格增强的全局导航示意图。

如图 3 所示，每一步中，深度强化学习智能体都会在运动前预测路径类型。当深度强化学习模型预测为原子移动时，它还需要从 16 个朝向中选取一个，每个朝向之间间隔固定角度。随后，智能体按照指定朝向向前移动固定距离。当模型选择使用导航网格（图 3）时，它将搜索智能体与目标之间的最优路径，并

challenges as modern 3D FPS games often simulate firearm recoil, resulting in realistic bullet trajectories. This becomes a training challenge for the agent, as it must learn how to control the bullet trajectory to hit opponents accurately. This section will discuss how to train a globally interactive game on a 1000×400 (unit: meters in UE4) map and the approaches adopted to address these two challenges. The results of these approaches will be demonstrated in the section IV.

A. Framework

We illustrate our framework as shown in fig. 2. In each step, the game's basic information is separately input into a feature extraction module. All scalar information undergoes feature extraction through a feed-forward network. The depth map and the Lindar [8] is input into the feature extraction module. By utilizing two-dimensional convolution to perceive the contour features of objects represented in the depth map and employing one-dimensional circular convolution to extract surrounding terrain features, we achieve a multi-modal fusion-based environmental perception approach by fusing the features from each feature extraction module with the environment information of the game after concatenation.

To give the model memory capacity, we utilized LSTM (Long Short-Term Memory) networks. The features filtered through the LSTM are then input into the policy network and value network, which are used for action prediction and value calculation respectively. As mentioned in section II-B, we divided the action space into nine action heads based on human operation. Each action head is represented by a one-hot vector that indicates the execution state of the corresponding action. There are certain dependencies among the action heads. For example, according to the limitations of the game itself, the model should not predict both firing and movement simultaneously. To decouple the dependencies among actions, we did not adopt the approach of parallel prediction for action heads. Instead, we sequentially output the action heads using a hierarchical action mask with auto-regressive embedding [11].

In typical reinforcement learning tasks, action masking is employed to block corresponding actions in specific states to enhance exploration efficiency. In our policy network, all action heads are sequentially generated. Except for the first action head, the output of each subsequent action head is determined jointly by the embedding of the previous action and the policy network's output. Through auto-regressive embedding, the policy implicitly learns the dependencies between preceding and succeeding actions and propagates them layer by layer. During action prediction, the policy can temporarily mask actions in subsequent layers that conflict with the current action. The policy also determines whether to mask the actions in the current layer based on the preceding actions. In this way, we conduct hierarchical action masking through auto-regressive prediction, ensuring action compliance while improving exploration efficiency. After all action heads have completed their outputs, these actions are delivered to the game client for execution.

B. Navigation Mesh and Shooting-rule Enhanced Reinforcement Learning

To address the challenges of global navigation on a 1000×400 (unit: meters in UE4) map and the issue of firing when encountering enemies in-game, we employ Navigation Mesh And Shooting-rule enhanced Reinforcement Learning (NSRL). This approach combines the integration of a Navigation Mesh (Navmesh) and atomic shooting rules to enhance the performance of the game's AI.

Global Navigation. In the context of global navigation, simply incorporating Navmesh directly into the program would lead to rule-based behavior in game AI, thereby sacrificing the diversity provided by DRL models. Therefore, in NSRL, a more gentle approach is employed. The decision-making power to use the Navmesh is delegated to the DRL model, allowing the model to predict whether to enable the Navmesh. By setting Navmesh as a predictable action, NSRL maintains the ability for global navigation while still preserving the diversity of the game AI provided by the DRL model.

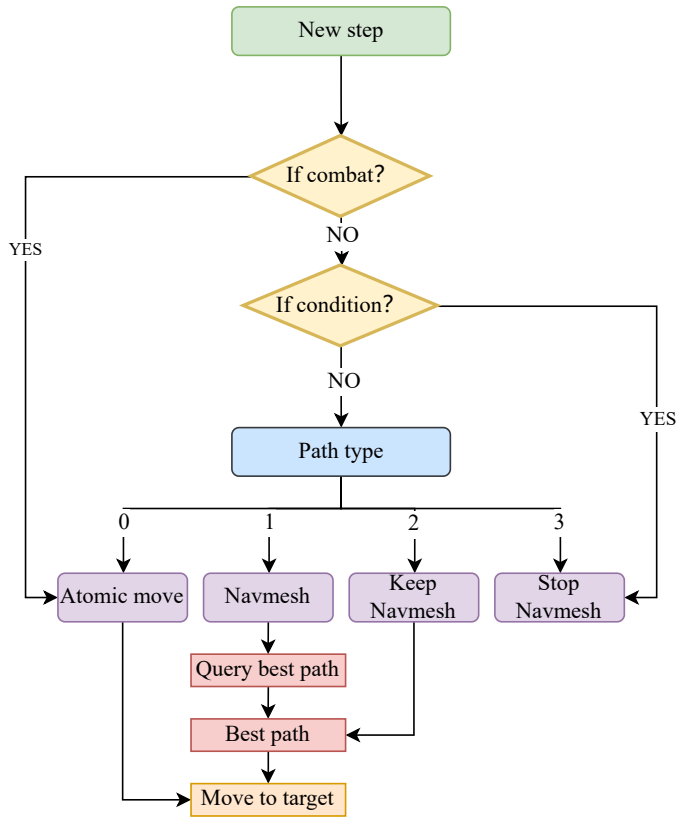


Fig. 3. The illustration of Navmesh enhanced global navigation.

As fig. 3 shown, in each step, the DRL agent will predict the Path type before motion. When the DRL model predicts atomic movement, it also needs to select an orientation from a set of 16 orientations, with each orientation spaced at a fixed angle interval. Subsequently, the agent moves forward a fixed distance according to the specified orientation. When the model chooses to use the Navmesh (fig. 3), it searches for the optimal path between the agent and the target using the

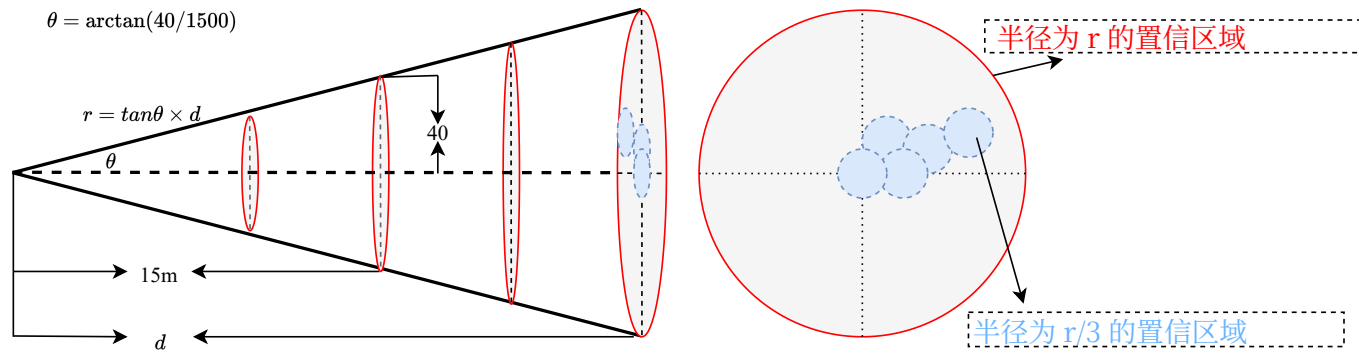


图 4. 射击规则示意图。

沿着该路径移动。路径类型中包含一个保持上一导航网格的动作，旨在当目标静止时避免重复调用导航网格，从而减少资源消耗。

在整个游戏过程中持续调用导航网格是不可行的，因为这样将无法与直接使用导航网格进行移动区分开来。为了解决这个问题，我们将整个游戏划分为多个时间片。在固定的时间间隔内，客户端会检查是否有任何使用导航网格的请求。一旦检测到使用导航网格的请求，深度强化学习模型对其他路径类型的预测将被屏蔽，智能体将执行导航网格路径，直至到达目标、遭遇敌人或达到导航网格使用的时间限制。这种方法在确保全局导航的同时，降低了资源消耗，并保留了深度强化学习模型固有的多样性。需要注意的是，当智能体处于战斗场景时，通常意味着对手对于智能体是可见的。因此，为了确保战斗的流畅性，优先使用原子移动而非导航网格。导航网格通常是在追击敌方目标时进行预测。

射击规则。在Arena Breakout中，由于模拟了真实的枪械弹道和后坐力，NSRL只需预测枪管方向以及是否开火即可确保射击控制。这种方法具有保持稳定射击的优势，同时增强了射击动作的多样性。在作战场景中，基准射击点是根据对手身体各部位的可见优先级来确定的。系统会计算一个射击置信区域，该区域定义了随机射击的范围，而超出置信区域的射击将被截断。如图4所示，我们设定了 $\arctan(40/1500)$ 作为基准角度，并根据基准半径计算不同距离下的置信区域半径 r 。此外，我们在连续两次射击之间额外引入了一个置信区域，该区域由半径为 $r/3$ 的圆表示。这种设计模仿了人类玩家在游戏中控制后坐力的行为，模拟了“点射”的概念，即在一定范围内进行射击以保持精度。

C. 奖励设计

在奖励设计中，确保智能体对全局导航

和射击有全面理解至关重要。因此，在设计奖励时，需要综合考虑全局导航、射击性能以及最终奖励。由于PMCA主要用于高级别比赛，它需要展现出一定的实力。给定一个时间范围 T ，最终奖励 r_T 可以定义如下：

$$r_T = \begin{cases} 20, & \text{if win,} \\ -20, & \text{if lose,} \\ -25, & \text{if draw,} \end{cases}$$

导航奖励 r_d 为 $r_d = \Delta_d \times 0.05$ ，其中 Δ_d 是智能体与对手之间的距离。我们为智能体的射击行为增加了额外的辅助奖励 r_{aux} ，以增强其拟人化特征。这些奖励根据专家先验分为战斗和移动两类，确保智能体不会出现令玩家困惑的行为。最终奖励 r 定义为：

$$r = r_T + r_d + r_{aux}$$

D. 训练过程

为了在大型 1000×400 （单位：米，UE4 尺度）地图上训练智能体，我们首先将整张地图划分为八个区域（图 5 中的红色方块）。每局比赛开始时，智能体和对手会在同一区域内随机生成，且双方生成点之间必然存在掩体或障碍物。当双方相遇时，只要一方能观察到另一方，战斗随即开始。这种方法不仅训练了智能体的战斗能力，还通过鼓励使用非战斗战术击败对手，提升了其战略决策能力。在训练过程中，当任意一方阵亡或时间达到超时限制时，该回合结束。

我们的系统采用 PPO 进行策略改进。我们采用重要性加权演员-学习者架构（IMPALA）[12] 算法作为训练框架中的主要算法，使智能体能够从多种经验来源中学习。智能体与环境交互以收集数据。经过若干回合后，智能体获取的数据用于估计优势

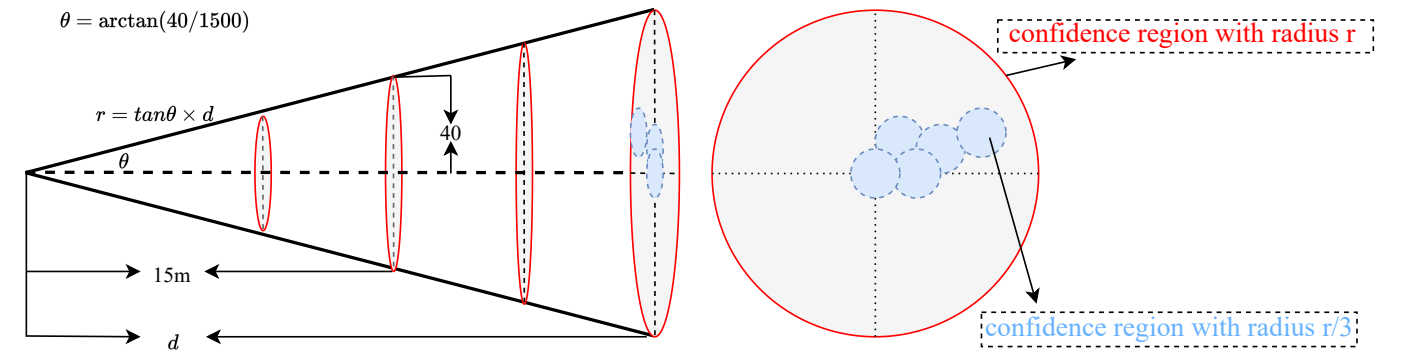


Fig. 4. The illustration of shoot rules.

Navmesh and moves along that path. The Path type includes an action to keep the previous Navmesh, which aims to avoid re-calling the Navmesh when the target is stationary, reducing resource consumption.

It is not feasible to call the Navmesh throughout the entire game as it would be indistinguishable from directly using Navmesh for movement. To address this, we divide the entire game into time slices. At fixed intervals, the client will check for any requests to use the Navmesh. Once a request to use the Navmesh is detected, the predictions of the DRL model for other Path types are masked, and the agent executes the Navmesh until it reaches the target, encounters an enemy, or reaches the time limit for Navmesh usage. This approach ensures global navigation while reducing resource consumption and preserving the diversity inherent in DRL models. Note that when the agent is in a combat scenario, it typically implies that the opponent is visible to the agent. Therefore, to ensure smooth combat, atomic movement is preferred over Navmesh usage. Navmesh is usually predicted when pursuing enemy targets.

Shooting Rules. In Arena Breakout, where realistic firearm ballistics and recoil are simulated, NSRL only needs to predict the gun barrel direction and whether to fire to ensure shooting control. This approach offers the advantage of maintaining stable shooting while enhancing the diversity of shooting actions. In combat scenarios, the baseline firing point is determined based on the visibility priority of the opponent's body parts. A shooting confidence region is calculated that defines the area within which random shots are fired, while shots outside the confidence region are truncated. As shown in fig. 4, We have set $\arctan(40/1500)$ as the baseline angle, and the confidence region radius r for different distances is calculated based on the baseline radius. Additionally, we have incorporated an extra confidence region between two consecutive shots, which is represented by a circle with a radius of $r/3$. This imitates the behavior of human players controlling recoil in the game, simulating the concept of "burst shooting" where shots are fired within a certain range to maintain accuracy.

C. Reward Design

In reward design, it is important to ensure that the agent has a comprehensive understanding of both global navigation

and shooting. Therefore, when designing the rewards, it is necessary to consider global navigation, shooting performance, and the final reward. Since the PMCA is primarily used in high-level matches, it needs to exhibit a certain level of strength. Given a horizon T , the final reward r_T can be defined as follows:

$$r_T = \begin{cases} 20, & \text{if win,} \\ -20, & \text{if lose,} \\ -25, & \text{if draw,} \end{cases}$$

The navigation reward r_d is $r_d = \Delta_d \times 0.05$. Where Δ_d is the distance between agent and opponent. We have implemented additional auxiliary rewards r_{aux} for the agent's shooting behavior to enhance its human-like characteristics. These rewards are categorized into two parts: combat and movement, based on expert priors. This ensures that our agent does not exhibit confusing behaviors that might perplex players. The final reward r defined as:

$$r = r_T + r_d + r_{aux}$$

D. Training Process

To train the agent on a large 1000×400 (unit: meters in UE4) map, we initially divided the entire map into eight regions (red square in fig. 5). At the beginning of each match, the agent and the opponent are randomly generated within the same region. There is guaranteed to be cover or obstacles between the spawn points of the two sides. When the two sides encounter each other, a battle begins as long as one side has a line of sight to the other. This approach not only trains the agent's combat abilities but also enhances its strategic decision-making by encouraging the use of non-combat tactics to defeat opponents. During the training process, an episode ends when either one of the players dies or the time reaches the timeout limit.

PPO is used in our system for policy improvement. We employed the importance-weighted actor-learner Architecture (IMPALA) [12] algorithm as the primary algorithm in our training framework, which enables agents to learn from multiple sources of experience. The agent interacts with the environment to collect data. After several episodes, the data obtained by the agent is used to estimate the advantage



图 5. ArenaBreakout 中的地图——农田。

函数。我们使用广义优势估计[13] 来估计每个时间步 t 的优势函数：

$$\hat{A} = \sum_{t=0}^T (\gamma\lambda)^t A_{t+1}^{\pi_\theta} \quad (2)$$

并且优势函数 A 定义如下：

$$A^{\pi_\theta}(s_t, a_t) = r(s_t, a_t) + \gamma V^{\pi_\theta}(S_{t+1}) - V^{\pi_\theta}(S_t) \quad (3)$$

其中 $a_t \sim \pi_\theta(\cdot|s_t)$ 。 $r(s_t, a_t)$ 是根据当前状态和动作的奖励函数。 V^{π_θ} 是价值函数，用于定义当前策略的状态价值。

在获得优势函数之后，我们可以计算策略的损失。我们使用带裁剪的 PPO，其损失函数定义如下：

$$\mathcal{L}^\pi(\theta) = \min[\rho(\theta)\hat{A}_t, clip(\rho(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}], \quad (4)$$

以约束采样策略与目标策略之间的距离。价值损失定义如下：

$$\mathcal{L}^v(\phi) = \mathbb{E}_{a \sim \pi_\theta} \left[\sum_{k=t}^T \gamma^{k-t} r(s_k, a_k) - V(s_t) \right]^2 \quad (5)$$

并且当前策略 π_θ 的总损失定义如下：

$$\mathcal{L}(\theta) = \mathcal{L}^\pi(\theta) - \alpha \mathcal{L}^v(\phi) + \beta \mathcal{L}^e \quad (6)$$

$\mathcal{L}^e$ 是策略的熵，用于增强探索。

IV. 实验

A. 实验设置

在本节中，我们将展示实验结果。我们将论证NSRL在全局导航、射击和最终胜率方面带来的改进。实验使用了8个GPU和3200个CPU核心，每次训练会话涉及4个演员、4个学习者和6000个客户端。所有实验均基于PPO算法。为确保公平性，所有训练会话使用了相同的资源集合。

B. 结果

全局导航。为了在相同条件下比较NSRL智能体与强化学习智能体的全局导航性能，我们进行了一项研究：将两个智能体在相同的环境中进行训练和测试。NSRL与强化学习智能体均使用相同的专家先验行为树作为各自的对手进行训练。实验中，起始点与游戏内资源保持一致。在此情景下，

我们记录了NSRL智能体与强化学习智能体在地图上的遍历点，并收集了超过1000场游戏会话的数据用于分析。NSRL与强化学习智能体的全局导航可视化如图6所示。从图6(a)和图6(b)可以明显看出，与强化学习相比，NSRL在x轴长度、y轴宽度和z轴高度上均拥有更广的运动范围。这表明，在相同条件下，得益于规则增强的NSRL有效提升了深度强化学习智能体的全局导航能力。事实上，NSRL智能体拥有更多样化的行为。这种多样性使NSRL智能体能够在环境中探索更广泛的区域，从而收集到更多有价值的经验。这一结果有助于我们设计出更具交互性的智能体，使其能够更有效地导航整个地图。

NSRL智能体的能力。为了展示NSRL相比纯RL的能力，两种智能体在训练过程中使用了相同的资源和对手（基于专家先验行为树）。我们提取了NSRL智能体和RL智能体在200个时间步时的胜率变化，并绘制在图7中。在相同时间步，NSRL的胜率平均稳定比RL智能体高出约30%。这突显了NSRL在适应专家先验行为树方面的卓越性能。此外，这也强调了NSRL更高的训练效率，因为它能在相同时间步内培养出能力更强的智能体。

NSRL的射击表现。由于现代3D FPS游戏中普遍使用真实枪支后坐力和弹道效果，深度强化学习智能体必须表现出不会让玩家困惑的类人射击行为。为了分析这一点，我们设计了一个特定的实验场景，其中NSRL智能体和强化学习智能体需要向固定目标位置射击。我们收集了目标位置附近弹着点分布的坐标，并将其整合到图8中。由图8(a)可知，当人类玩家射击时，其子弹会分散在中心目标点周围，这符合真实枪支后坐力和弹道效果的逻辑。图8(b)的结果显示，NSRL智能体的子弹分布呈现出与人类相似的趋势，但分散程度更高。这可能是由于模型对FPS游戏中“压枪控制”概念缺乏足够的理解。图8(c)说明，纯RL无法有效控制枪械，并可能表现出令人困惑的行为。

总之，NSRL 有效增强了深度强化学习智能体的全局导航能力，使其展现出更多样化的策略和行为。NSRL 在保持一定竞争力水平的同时提高了训练效率，使其成为部署在高水平场景中的必要工具。此外，NSRL 的射击规则增强提升了深度强化学习智能体的人性化程度，确保它们不会做出削弱玩家游戏体验的令人困惑的行为。



Fig. 5. The map in Arena Breakout, the farmland.

function. We use Generalized Advantage Estimation[13] to estimate the advantage function at each time step t :

$$\hat{A} = \sum_{t=0}^T (\gamma\lambda)^t A_{t+1}^{\pi_\theta} \quad (2)$$

And the advantage function A is defined as:

$$A^{\pi_\theta}(s_t, a_t) = r(s_t, a_t) + \gamma V^{\pi_\theta}(S_{t+1}) - V^{\pi_\theta}(S_t) \quad (3)$$

Where $a_t \sim \pi_\theta(\cdot|s_t)$. $r(s_t, a_t)$ is the reward function according to the current state and action. V^{π_θ} is the value function, which defines the state value of the current policy.

After having the advantage function, we can calculate the loss of the policy. We use PPO with clip, and the loss function is defined as:

$$\mathcal{L}^\pi(\theta) = \min[\rho(\theta)\hat{A}_t, clip(\rho(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}], \quad (4)$$

to constrain the distance between the sample policy and the target policy. The value loss is defined as:

$$\mathcal{L}^v(\phi) = \mathbb{E}_{a \sim \pi_\theta} \left[\sum_{k=t}^T \gamma^{k-t} r(s_k, a_k) - V(s_t) \right]^2 \quad (5)$$

And the total loss of current policy π_θ is defined as:

$$\mathcal{L}(\theta) = \mathcal{L}^\pi(\theta) - \alpha \mathcal{L}^v(\phi) + \beta \mathcal{L}^e \quad (6)$$

$\mathcal{L}^e$ is the entropy of policy which enhances the exploration.

IV. EXPERIMENT

A. Experimental Setup

In this section, we will present our experimental results. We will demonstrate the improvements brought by NSRL in global navigation, shooting, and final win rate. The experiments were conducted using 8 GPUs and 3200 CPU cores, with 4 actors, 4 learners, and 6000 clients involved in each training session. All experiments were based on the PPO algorithm. To ensure fairness, all training sessions used the same set of resources.

B. Results

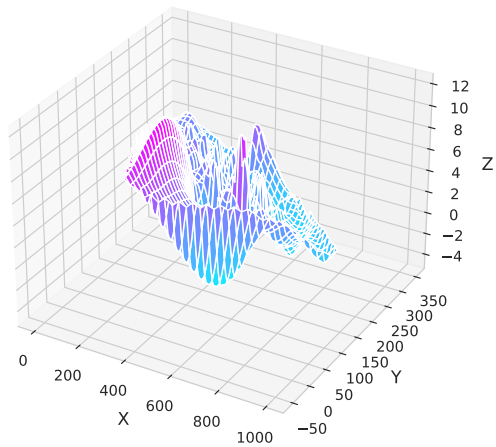
Global Navigation. To compare the global navigation performance of the NSRL agent and the RL agent under identical conditions, we conducted a study where both agents were trained and tested in the same environment. The NSRL and RL agents were trained using the same expert prior behavior tree as their opponent. The starting points and in-game resources were kept consistent across the experiments. In this scenario,

we recorded the traversal points of both the NSRL agent and the RL agent on the map. We collected data from over 1000 game sessions for analysis. The NSRL and RL agent global navigation visualization is shown in fig. 6. From the fig. 6(a) and fig. 6(b), it is evident that NSRL has a wider motion range in terms of the length on the x-axis, breadth on the y-axis, and height on the z-axis compared to RL. This indicates that under the same conditions, NSRL, which benefits from rule enhancement, effectively improves the global navigation capability of the DRL agent. Indeed, NSRL agent possesses a more diverse range of behaviors. This diversity allows the NSRL agent to explore a wider range of areas in the environment, enabling it to gather more valuable experiences. This result assists us in designing more interactive agents that can navigate the entire map more effectively.

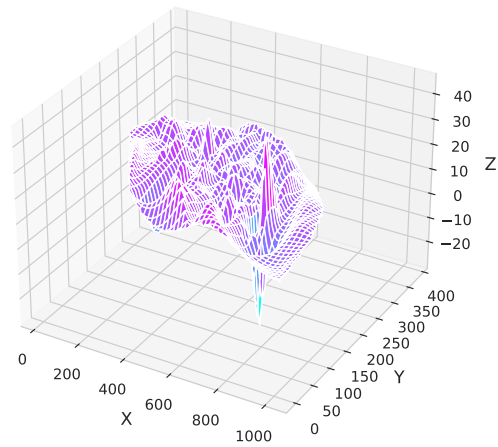
Competence of NSRL agent. To demonstrate the competence of NSRL compared to pure RL, both agents were trained using identical resources and opponents (based on expert prior behavior trees) throughout the training process. We extracted the win rate changes of the NSRL agent and RL agent at 200 timesteps and plotted them in fig. 7. At the same timestep, the win rate of NSRL was consistently around 30% higher on average compared to the RL agent. This highlights the superior performance of NSRL in adapting to expert prior behavior trees. Furthermore, it emphasizes the improved training efficiency of NSRL, as it is capable of producing more competent agents at the same timestep.

Shooting of NSRL. Due to the prevalent use of realistic firearm recoil and ballistic effects in modern 3D FPS games, it is crucial for DRL agents to exhibit human-like shooting behavior that does not confuse players. To analyze this, we designed a specific experimental scenario where the NSRL and RL agents were tasked with shooting at fixed target locations. We collected the coordinates of bullet impact patterns near the target locations and integrated them into fig. 8. From the fig. 8(a), it can be observed that when human players shoot, their bullets are dispersed around the central target point, which aligns with the logic of realistic firearm recoil and ballistic effects. The result in fig. 8(b) shows the bullet distribution of the NSRL agent exhibits a similar trend to that of humans but with a higher degree of dispersion. This may be due to a lack of sufficient understanding of the concept of "spray control" in FPS games within the model. fig. 8(c) illustrates that pure RL alone cannot effectively control firearms and may exhibit perplexing behaviors.

In summary, NSRL effectively enhances the global navigation capabilities of DRL agents, enabling them to exhibit more diverse strategies and behaviors. NSRL improves training efficiency while maintaining a certain level of competitiveness, making it a necessary asset for deployment in high-level scenarios. Moreover, NSRL's shooting rule augmentation enhances the human-like nature of DRL agents, ensuring that they do not exhibit confusing behaviors that diminish the players' gaming experience.



(a) 强化学习全局导航可视化



(b) NSRL全局导航可视化

图6. 强化学习与NSRL全局导航的可视化。颜色越接近紫色，表示遍历次数越多；越接近蓝色，则遍历次数越少。NSRL的可视化显示，大部分区域接近紫色，表明遍历次数较多，而接近蓝色的区域较少。相比之下，强化学习的可视化显示只有一小部分区域接近紫色，这表明强化学习在全局导航能力上存在局限。

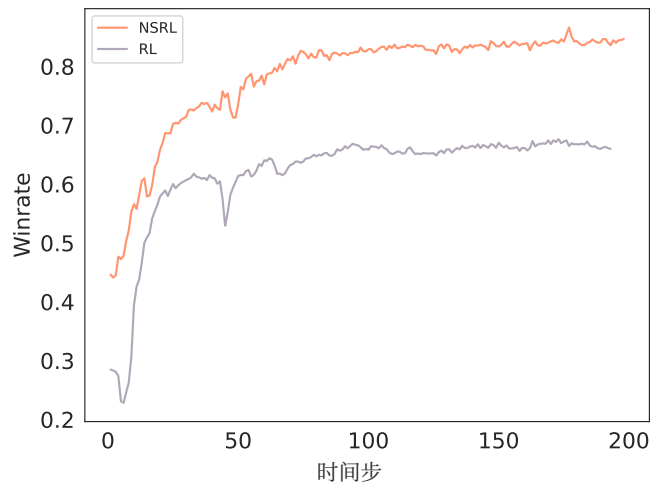


图7. 训练过程中强化学习智能体和NSRL智能体的胜率随BT变化趋势。

V. 相关工作

A. 第一人称射击游戏中的强化学习

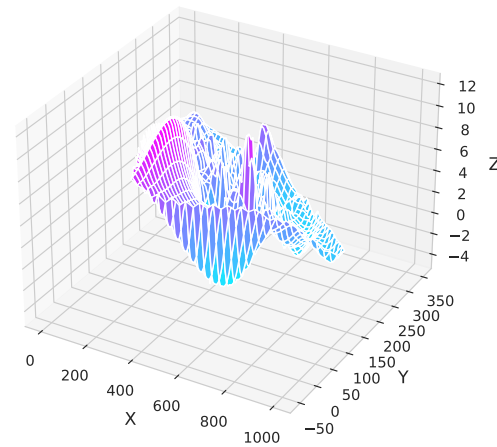
在强化学习领域，第一人称射击游戏是一个极具价值的研究环境。在第一人称射击游戏中，需要在部分观测下平衡移动和射击动作，并在竞争环境中实现最佳性能。目前，许多第一人称射击游戏被用于强化学习算法的研究。两款90年代的游戏很早就被引入强化学习环境。Kempka 等人 [14] 将 ViZDoom 引入，Jaderberg 等人 [3] 则将雷神之锤 III 竞技场构建为强化学习环境。基于这些环境，目前已经发表了许多优秀成果。Hosu 和 Rebedea [9], 提出的 DRQN 是 ViZDoom 中最有影响力的工作之一。DRQN 是

还实现了模块化，允许针对游戏的不同阶段独立训练不同的模型，在死亡竞赛场景中，其表现显著优于游戏内置的 AI 智能体以及普通人类玩家。Wu 和 Tian [1] 使用演员-评论家课程学习来训练基于视觉的智能体，并在 ViZDoom 中赢得了 2016 年 ViZDoom 人工智能竞赛第一赛道的冠军。Jaderberg 等人 [3] 首次证明，在流行的 3D 多人第一人称视频游戏《雷神之锤 III 竞技场夺旗模式》(28) 中，仅使用像素和游戏分数作为输入，智能体即可达到人类水平的表现。这一成果通过一种新颖的两层优化流程实现，在该流程中，大量独立的强化学习智能体通过数千场并行比赛同时进行训练，这些智能体在随机生成的环境中组队协作并相互对抗。

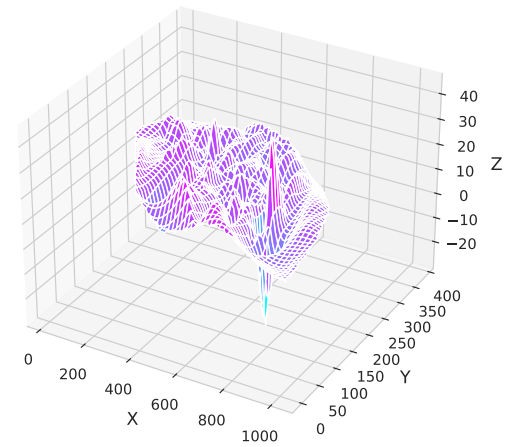
除了 ViZDoom 和《雷神之锤 III 竞技场》，Pearce 和 Zhu[4] 采用行为克隆的方法训练了一个智能体在 CS: GO 中玩死亡竞赛模式。他们的方法展示了相当不错的性能和较高的数据效率，为强化学习在第一人称射击游戏中的应用开辟了新途径。Chen 等人 [5] 开发了 WILD-SCAV，这是一个基于 3D 开放世界第一人称射击游戏的强大且可扩展的环境，该环境具有更高的多样性和复杂性。他们希望弥合现有环境难以扩展到更复杂问题之间的差距。Zhao 等人针对三项任务提出了一种规则增强深度强化学习算法。他们的智能体在多个基于规则和基于强化学习的智能体中表现更优。

VI. 结论

在本文中，我们提出了一种适用于 Arena Breakout 的全地图交互智能体 PMCA，该智能体解决了将深度强化学习应用于现代第一人称射击游戏时面临的全局导航与真实弹道行为挑战。为应对这些挑战，我们提出了一种创新方法，将



(a) Visualization of RL Global Navigation



(b) Visualization of NSRL Global Navigation

Fig. 6. Visualization of RL and NSRL Global Navigation. The closer the color is to purple, the higher the number of traversals, while the closer it is to blue, the fewer the number of traversals. The NSRL visualization shows that most areas are closer to purple, indicating a higher number of traversals, while there are fewer areas close to blue. On the other hand, the RL visualization reveals that only a small portion is close to purple, suggesting limitations in RL's global navigation capabilities.

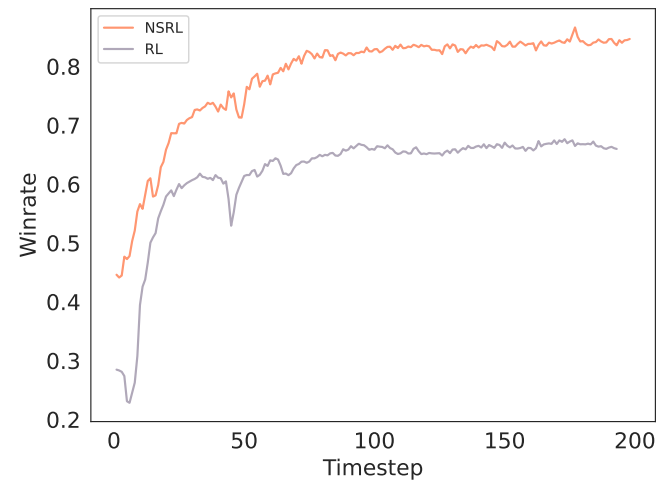


Fig. 7. The trend of win rates verses BT changes during the training process for the RL agent and NSRL agent.

V. RELATED WORK

A. Reinforcement Learning in FPS Games

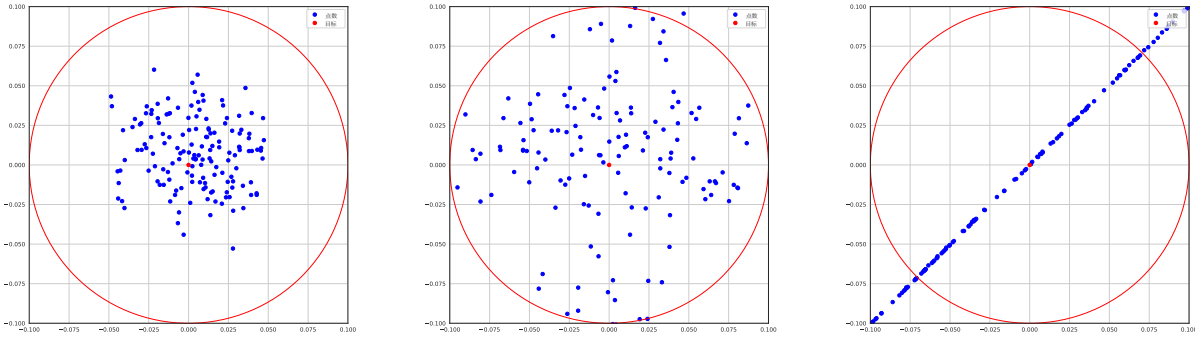
In the field of reinforcement learning, FPS is a highly valuable research environment. In FPS games, it is necessary to balance both movement and firing actions under partial observation and achieve the best performance in a competitive environment. Currently, many FPS games are used for research on reinforcement learning algorithms. Two 1990s games were introduced to the RL environment in the early stage. Kempka et al. [14] introduced ViZDoom and Jaderberg et al. [3] built Quake III Arena as RL environment. There have been many excellent works published based on these environments currently. The DRQN, proposed by Hosu and Rebedea [9], is one of the most influential works in ViZDoom. DRQN is

also modularized to allow different models to be independently trained for different phases of the game which substantially outperforms built-in AI agents of the game as well as average humans in deathmatch scenarios. Wu and Tian [1] using Actor-Critic curriculum learning to train vision-based agents in ViZDoom and win the champion of Track1 in ViZDoom AI Competition 2016. Jaderberg et al. [3] demonstrate for the first time that an agent can achieve human-level in a popular 3D multiplayer first-person video game, Quake III Arena Capture the Flag (28), using only pixels and game points as input. The result is achieved by a novel two-tier optimization process in which a population of independent RL agents is trained concurrently from thousands of parallel matches with agents playing in teams together and against each other on randomly generated environments.

Besides ViZDoom and Quake III Arena, Pearce and Zhu [4] learn an agent to play deathmatch in CS: GO with adopting behavioral cloning. Their method shows reasonably good performance and high data efficiency, a new way to apply RL in FPS games. Chen et al. [5] develop WILD-SCAV, a powerful and extensible environment based on a 3D open-world FPS game which is an environment with greater diversity and complexity. They want to bridge the gap that the existing environment is hardly extensible to more complicated problems. Zhao et al. propose a rule-enhanced deep reinforcement learning algorithm for three tasks. And their agent has better performance over multiple rule-based and RL-based agents.

VI. CONCLUSION

In this paper, we propose a full-map interactive agent, PMCA, for Arena Breakout that addresses the challenges of global navigation and realistic ballistic behavior inherent in applying DRL to modern FPS games. To address these challenges, we propose a novel method that combines



(a) 人类玩家的子弹分布 (b) NSRL智能体的子弹分布 (c) 强化学习智能体的子弹分布

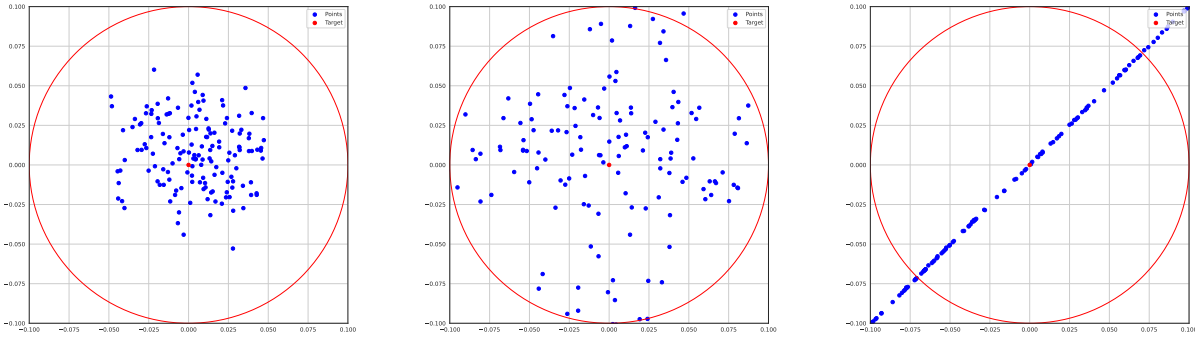
图 8. 人类玩家、NSRL智能体与强化学习智能体的子弹分布对比。

导航网格与NSRL相结合。我们采用射击规则约束来解决类人行为问题，同时利用导航网格增强全局移动能力。通过实验验证了NSRL的优越性。该智能体自2024年初已部署至Arena Breakout，标志着强化学习在游戏领域实际应用的重要里程碑。在未来的工作中，我们将通过引入占用地图与不同计算机视觉技术，进一步探索智能体在大规模3D环境中的探索能力。

参考文献

[1]吴宇星和田远东. 基于演员-评论家课程学习训练第一人称射击游戏智能体. 在国际学习表征会议, 2016.[2] Guillaume Lample 和 Devendra Singh Chaplot. 使用深度强化学习游玩第一人称射击游戏. 在 *AAAI* 人工智能大会论文集, 第31卷, 2017.[3] Max Jaderberg, Wojciech M Czarnecki, Iain Dunning, Luke Marris, Guy Lever, Antonio Garcia Castaneda, Charles Beattie, Neil C Rabinowitz, Ari S Morcos, Avraham Ruderman 等人. 基于种群的强化学习在3D多玩家游戏中达到人类水平表现. *科学*, 364(6443):859–865, 2019.[4]Tim Pearce 和朱军. 利用大规模行为克隆的反恐精英死斗模式. 在 *2022年IEEE游戏大会(CoG)*, 第104–111页. IEEE, 2022.[5] 陈曦、史天宇、赵清鹏、孙玉晨、高云飞和王向军. Wild-scav: 基于Unity3D环境FPS游戏人工智能基准测试. *arXiv预印本 arXiv:2210.09026*, 2022.[6] Greg Snook. 使用导航网格的简化3D移动与路径查找. 载于 Mark DeLoura 编辑的游戏编程精粹, 第288–304页. Charles River Media, 2000.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford 和 Oleg Klimov. 近端策略优化算法. *arXiv预印本 arXiv:1707.06347*, 2017.[8]Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew 和 Igor Mordatch. 多智能体自动课程中的涌现工具使用. *arXiv预印本 arXiv:1909.07528*, 2019.[9]Ionel-Alexandru Hosu 和 Traian Rebedea. 使用深度强化学习和人类检查点回放玩 Atari游戏. *arXiv预印本 arXiv:1607.05077*, 2016.[10] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra 和 Martin Riedmiller. 确定性策略梯度算法. 在国际机器学习大会, 第387–395页. PMLR, 2014.[11] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev 等人. 利用多智能体强化学习在星际争霸II中达到宗师级别. *自然*, 575(7782):350–354, 2019.[12] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning 等人. IMPALA: 基于重要性加权演员-学习者架构的可扩展分布式深度强化学习. 在国际机器学习大会, 第1407–1416页. PMLR, 2018.[13] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan 和 Pieter Abbeel. 使用广义优势估计的高维连续控制. *arXiv预印本 arXiv:1506.02438*, 2015.[14] Michał Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek 和 Wojciech Jaśkowski. VizDoom: 基于毁灭战士的视觉强化学习AI研究平台. 在 *2016年IEEE计算智能与游戏会议(CIG)*, 第1–8页. IEEE, 2016.



(a) Bullet Distribution of Human Players (b) Bullet Distribution of NSRL agent (c) Bullet Distribution of RL agent

Fig. 8. Bullet Distribution of Human Players, NSRL agent, and RL agent.

Navmesh 和 NSRL. The shooting-rule constraint is used to address the human-like behavior problem, while the Navmesh enhances global movement capabilities. Through experiments, we demonstrate the superiority of NSRL. This agent has been deployed to Arena Breakout since early 2024, marking an important milestone in the practical application of reinforcement learning in gaming. In our future work, we will further investigate the agent’s ability to explore large-scale 3D environments by adopting occupancy maps and different computer vision techniques.

REFERENCES

[1] Yuxin Wu and Yuandong Tian. Training agent for first-person shooter game with actor-critic curriculum learning. In *International Conference on Learning Representations*, 2016.

[2] Guillaume Lample and Devendra Singh Chaplot. Playing fps games with deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

[3] Max Jaderberg, Wojciech M Czarnecki, Iain Dunning, Luke Marris, Guy Lever, Antonio Garcia Castaneda, Charles Beattie, Neil C Rabinowitz, Ari S Morcos, Avraham Ruderman, et al. Human-level performance in 3d multiplayer games with population-based reinforcement learning. *Science*, 364(6443):859–865, 2019.

[4] Tim Pearce and Jun Zhu. Counter-strike deathmatch with large-scale behavioural cloning. In *2022 IEEE Conference on Games (CoG)*, pages 104–111. IEEE, 2022.

[5] Xi Chen, Tianyu Shi, Qingpeng Zhao, Yuchen Sun, Yunfei Gao, and Xiangjun Wang. Wild-scav: Benchmarking fps gaming ai on unity3d-based environments. *arXiv preprint arXiv:2210.09026*, 2022.

[6] Greg Snook. Simplified 3d movement and pathfinding using navigation meshes. In Mark DeLoura, editor, *Game Programming Gems*, pages 288–304. Charles River Media, 2000.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[8] Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. *arXiv preprint arXiv:1909.07528*, 2019.

[9] Ionel-Alexandru Hosu and Traian Rebedea. Playing atari games with deep reinforcement learning and human checkpoint replay. *arXiv preprint arXiv:1607.05077*, 2016.

[10] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. Pmlr, 2014.

[11] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

[12] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International conference on machine learning*, pages 1407–1416. PMLR, 2018.

[13] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

[14] Michał Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek, and Wojciech Jaśkowski. Vizdoom: A doom-based ai research platform for visual reinforcement learning. In *2016 IEEE conference on computational intelligence and games (CIG)*, pages 1–8. IEEE, 2016.